SWI C01 — Reservation System

Rezervace nabíjecích stanic pro elektromobily · tým SW-DEV-AJVZMPIA

Název týmu	SW-DEV-AJVZMPIA
Členové týmu	Viacheslav Zlobin (GitHub sk1drovv), Maksym Panfilov (Makusu1), Aleksej Južaninov (alexejjuzaninov), Ivan Akulovich (Vezem1r)
URL repozitáře	https://github.com/alexejjuzaninov/EC-reservationSystem
Rezervovaný resource	Nabíjecí stanice pro elektromobily (EV charging station)
Technologický stack	Python 3.12 + FastAPI + SQLAlchemy + PostgreSQL + pytest

Každé tvrzení v dokumentu odkazuje na soubor, issue nebo pull request v repozitáři, kde ho lze ověřit.

1 · Tým a repozitář

- Jeden veřejný repozitář na GitHubu pro celý tým.
- Všichni čtyři členové mají přístup a do repozitáře přispěli (GitHub Contributors: alexejjuzaninov, Makusu1, sk1drovv, Vezem1r).
- Struktura odpovídá zadání; složky, které přibyly v dalších cvičeních, jsou označené.

```
README.md                        přehled, rychlý start, rámec C01, walking skeleton
docs/
  intent-and-change.md           Project Frame, future pressure
  architecture-and-decisions.md  rozhodnutí o stacku (C01), architektura (C03), DD (C05)
  evidence-and-evolution.md      evidence spike (C01) a dalších cvičení
  specification.md               specifikace (C02)
spike/                           engineering spike C01: SQLAlchemy + PostgreSQL, pytest
src/                             walking skeleton CP1: FastAPI + SQLAlchemy + PostgreSQL (od C03)
```

2 · Reservation domain

Rezervujeme časové sloty na firemních nabíjecích stanicích pro elektromobily. Zaměstnanec si rezervuje okno předem, aby měl po příjezdu jistotu volné stanice.

Povinný prvek	Naše realizace
Resource	`ChargingStation`: id, connector_type (AC 22 kW / DC 50 kW), power_kw, location, active
Reservation	id, user_id, station_id, start_time, end_time, state, created_at
User	id, email, vehicle_type
States	DRAFT → CONFIRMED → CANCELLED (C02 později doplnila PENDING_APPROVAL, REJECTED, EXPIRED)
Operations	create, confirm, cancel, check availability
Common rule	Dvě CONFIRMED rezervace téže stanice se nesmí časově překrývat.
Domain rule	Uživatel smí mít nejvýše jednu CONFIRMED rezervaci za kalendářní den a ta smí trvat nejvýše 4 hodiny.
Boundary	Notification Service: e-mail při potvrzení a zrušení rezervace; je mimo naši kontrolu a může být pomalá nebo nedostupná.

Zdůvodnění vlastního pravidla

Nabíjecích stanic je málo a poptávka převyšuje kapacitu. Bez limitu by jeden uživatel zablokoval stanici na celý pracovní den. Pravidlo drží dostupnost pro celý tým a zároveň vytváří validační logiku, kterou lze testovat nezávisle na pravidle o překryvu.

3 · Project Frame

Obsah souboru `docs/intent-and-change.md`.

Reservation domain	Charging slots on company-owned EV charging stations.
Purpose	The system serves employees who drive electric vehicles to work and need a guaranteed charging slot during the working day. It prevents queueing and conflicts at a limited number of stations by reserving time windows in advance.
Users / Stakeholders	Employee — creates, confirms and cancels their own reservations. Facility admin — manages stations and may cancel any reservation.
Core concepts	Reservation, ChargingStation, User, TimeSlot.
Core operations	Create reservation (result: DRAFT) Confirm / approve reservation (DRAFT → CONFIRMED) Cancel reservation (DRAFT or CONFIRMED → CANCELLED) Check availability of a station within a time range
Persistent state	Reservation: id, user_id, station_id, start_time, end_time, state, created_at ChargingStation: id, connector_type, power_kw, location, active User: id, email, vehicle_type
State-changing operation	`confirm(reservation_id)`: DRAFT → CONFIRMED. The overlap check runs at this moment, not at creation time — a DRAFT is only an intent and may safely overlap with other drafts.
Common business rule	Confirmed reservations for the same resource must not overlap.
Domain-specific business rule	A user may hold at most one CONFIRMED reservation per calendar day, and its duration must not exceed 4 hours.
External / system boundary	Notification Service — sends an e-mail when a reservation is confirmed or cancelled. Accessed over HTTP. It is outside our control and may be slow or unavailable.
Assumption	We assume users cancel reservations they no longer need, so no-shows are rare and we do not need an automatic release mechanism yet.
Unknown	We do not know how the system should behave when the Notification Service is unavailable at the moment of confirmation — should the confirmation fail, or should it succeed and notify later?

4 · Selected future pressure

Category	R — Risk / Higher consequence
Concrete pressure	Systém se rozšíří z interního parkoviště na veřejné stanice s placenou rezervací. Dvojitá rezervace pak znamená, že platící zákazník přijede k obsazené stanici.
Why it is relevant	Dnes stojí překryv někoho pět minut čekání. Jakmile do hry vstoupí platba, stejná chyba znamená vracení peněz, stížnosti a ztrátu důvěry. Pravidlo o překryvu proto musí platit i při souběžných požadavcích a být pokryto automatickým testem, ne ponecháno na kontrole v UI.

V C01 se pressure neimplementuje. Určila otázku engineering spike (kapitola 6) a rozhodnutí, které z něj vyplynulo (kapitola 7).

5 · Change + review smyčka

Issue	#3 C01 engineering spike: vynutit pravidlo o překryvu i při souběhu
Autor změny	Viacheslav Zlobin, commity `6185d77` a `8639309` ve větvi `c01-db-overlap-constraint`; pull request otevřel Aleksej Južaninov
Reviewer	Maksym Panfilov (GitHub Makusu1), Pull Request #4, stav Approved před mergem
Obsah review	spike spuštěn lokálně (Docker Desktop, PostgreSQL 16, Python 3.13): `4 passed`; po dočasném zakomentování EXCLUDE constraintu 2 failed (`assert 2 == 1`, `DID NOT RAISE IntegrityError`), po vrácení opět `4 passed`; navazující interval projde
Integrace	PR #4 sloučen do `main` merge commitem `fd727bf`, issue #3 se tím uzavřel

6 · Engineering spike — varianta A (Persistence)

Rezervace se uloží do skutečné databáze PostgreSQL, znovu se načte a ověří se vynucení pravidla o překryvu. Kód je ve složce `spike/`, postup v `README.md` (Rychlý start).

Co bylo spuštěno

```
docker run --name swi-db -e POSTGRES_PASSWORD=swi -e POSTGRES_DB=reservations -p 5432:5432 -d postgres:16
docker exec swi-db psql -U postgres -c "CREATE DATABASE reservations_test;"
cd spike
py -3.12 -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python -m pytest -v
```

Testované scénáře

- `test_overlap.py`: rezervace A 09:00–11:00 potvrzena; B 10:00–12:00 na téže stanici skončí `OverlapError`; C 11:00–13:00 navazuje na A a je potvrzena. Nové načtení v jiné session ověří, že A a C zůstaly CONFIRMED a B DRAFT.
- `test_concurrency_risk.py`: dvě potvrzení překrývajících se rezervací ve dvou vláknech; bariéra zajistí, že obě dokončí kontrolu překryvu dřív, než kterékoli zapíše. Očekávána právě jedna CONFIRMED.
- `test_db_constraint.py`: databáze sama odmítne překrývající se CONFIRMED řádky i bez service vrstvy, navazující CONFIRMED a překrývající se DRAFT povolí.

Pozorovaný výsledek

Před změnou: sekvenčně kontrola v service vrstvě překryv zachytila, ale při souběhu skončily **obě** rezervace CONFIRMED (`AssertionError: assert 2 == 1`). Databáze sama žádné omezení neměla.

Po změně (commit `6185d77`), Windows 11, Python 3.12.8, PostgreSQL 16, SQLAlchemy 2.1.6, psycopg 3.3.6, pytest 9.1.1:

```
tests/test_concurrency_risk.py::test_two_concurrent_confirmations_cannot_both_succeed PASSED [ 25%]
tests/test_db_constraint.py::test_database_rejects_overlapping_confirmed_rows_even_without_service PASSED [ 50%]
tests/test_db_constraint.py::test_database_allows_adjacent_confirmed_and_overlapping_drafts PASSED [ 75%]
tests/test_overlap.py::test_overlapping_confirmation_is_rejected_and_state_persists PASSED [100%]
============================== 4 passed in 0.30s ==============================
```

Kontrola, že testy pravidlo skutečně ověřují

V dočasné kopii složky jsme kód záměrně pokazili a spustili stejné testy:

Úprava kódu	Výsledek	Co z toho plyne
bez EXCLUDE constraintu	2 failed: `assert 2 == 1`, `DID NOT RAISE IntegrityError`	testy odhalí chybějící ochranu při souběhu i na úrovni DB
uzavřený interval `'[]'` místo `'[)'`	2 failed (`ExclusionViolation` u navazující rezervace)	testy odhalí příliš přísné pravidlo, navazující časy jsou povolené

7 · Evidence a decision

Obsah souboru `docs/evidence-and-evolution.md`, část C01 Engineering Spike.

Question / unknown	Udrží kontrola překryvu v service vrstvě pravidlo BR-02 (potvrzené rezervace téže stanice se nepřekrývají), i když dvě potvrzení proběhnou souběžně?
What we did	PostgreSQL 16 v Dockeru, entity v SQLAlchemy a service vrstva s kontrolou překryvu. `test_overlap.py` ověří sekvenční případ a perzistenci po novém načtení v jiné session. `test_concurrency_risk.py` pustí dvě potvrzení ve dvou vláknech tak, aby obě dokončila kontrolu před zápisem. Spuštěno přes pytest.
Observed result	Sekvenčně: překryv odmítnut, navazující rezervace potvrzena, stav po novém načtení trvá. Souběžně: obě rezervace skončí CONFIRMED (`assert 2 == 1`). Kontrola v aplikaci pravidlo při souběhu neudrží, protože databáze sama žádné omezení nemá.
Decision / what changes	BR-02 vynucuje databáze: EXCLUDE constraint `no_overlapping_confirmed` nad `(station_id, tstzrange(start_time, end_time, '[)'))` pro řádky ve stavu CONFIRMED. Kontrola v service vrstvě zůstává kvůli srozumitelné chybě v běžném případě; porušení constraintu service převede na `OverlapError`. Stejné řešení převzal walking skeleton (ADR-01 v C03). Souvisí s pressure R.

Nález po integraci

Při opakovaném spouštění test souběhu občas selhal (3 z 50 běhů): PostgreSQL jeden ze dvou současných zápisů ukončil chybou `DeadlockDetected` místo porušení constraintu. Invariant platil, ale volající dostal neošetřenou chybu. Oprava: zápis se po deadlocku opakuje a skončí `OverlapError`; potom 50 z 50 běhů prošlo (issue #6, commit `eb8c636`).

8 · CP1 walking skeleton

Definice z C01 v `README.md`, část Walking skeleton pro CP1:

```
POST /reservations
  → validate      (Pydantic schema + business rules)
  → persist       (SQLAlchemy → PostgreSQL)
  → return reservation ID
  → automated check (pytest: POST vrátí 201 a ID, záznam je v DB)
```

Stav: implementováno v C03 ve složce `src/` (commit `bb299e5`). Automatická kontrola je `src/tests/test_walking_skeleton.py`. Skeleton nakonec obsahuje všech pět operací z C02 a celá sada testů ve složce `src/` končí výsledkem `71 passed`.

9 · Stack decision

Obsah souboru `docs/architecture-and-decisions.md`, část C01.

We chose Python 3.12 + FastAPI + SQLAlchemy + PostgreSQL + pytest instead of the default Java / Spring Boot stack. Reason: the team already works with Python, while no member has practical experience with Spring. A familiar stack lets us spend C01 on the reservation domain and on the engineering spike rather than on framework setup.

Trade-off: we support this stack ourselves, without the teacher's reference setup. FastAPI gives us request validation (Pydantic) and an OpenAPI schema out of the box, which covers what the CP1 walking skeleton needs. PostgreSQL is kept from the recommended stack, so the persistence layer stays comparable to other teams.

10 · Definition of Done

#	Položka	Stav	Kde je doložena
1	tým 3–4 členové	**splněno**	čtyři členové, titulní blok a kapitola 1
2	společný repo	**splněno**	`alexejjuzaninov/EC-reservationSystem`, přispěli všichni čtyři
3	jasný reservation domain	**splněno**	kapitola 2; Project Frame, Reservation domain
4	Resource + Reservation + User	**splněno**	kapitola 2; Project Frame, Persistent state
5	meaningful Reservation states	**splněno**	DRAFT → CONFIRMED → CANCELLED; Project Frame, State-changing operation
6	create + confirm/approve + cancel + availability	**splněno**	Project Frame, Core operations
7	common overlap rule	**splněno**	Project Frame, Common business rule
8	1 domain-specific business rule	**splněno**	denní limit a nejvýše 4 h; Project Frame, Domain-specific business rule
9	1 external/system boundary	**splněno**	Notification Service; Project Frame, External / system boundary
10	kompletní Project Frame	**splněno**	`docs/intent-and-change.md`, všechny položky vyplněné
11	1 Q/C/R/L future pressure	**splněno**	R — Risk / Higher consequence; kapitola 4
12	1 reviewed and integrated change	**splněno**	issue #3, PR #4 schválený Makusu1, merge `fd727bf`; kapitola 5
13	1 executed engineering spike	**splněno**	`spike/`, `4 passed`; kapitola 6
14	spike evidence + decision	**splněno**	`docs/evidence-and-evolution.md`, C01 Engineering Spike; kapitola 7
15	definovaný CP1 walking skeleton	**splněno**	`README.md`, Walking skeleton pro CP1; kapitola 8